INDIAN INSTITUTE OF TECHNOLOGY (IIT)JODHPUR

Assignment Report

on

"CHARACTER-LEVEL NAME GENERATION USING RNN VARIANTS"

Submitted By:
Aman Singh Negi
M25CSE004

Submitted To:
Dr. Anand Mishra
Assistant Professor
IIT Jodhpur

1. Introduction

A traditional and illuminating test of a neural network's capacity to learn sequential data is the generation of realistic human names character by character. Character-level models start with no prior linguistic knowledge, in contrast to word-level language models. In order to create strings that sound linguistically and culturally plausible, they have to start from scratch and learn the underlying phonotactics, which are the rules governing which vowels and consonants can sequentially combine.

This assignment's main goal is to use recurrent neural architectures to create and evaluate sequence models for character-level name generation. We implemented and assessed three different architectures in order to comprehend how various memory mechanisms affect a model's capacity to learn and produce these patterns:

1. **Vanilla Recurrent Neural Network (RNN):** This model, which serves as our baseline, emphasizes the fundamental mechanics (and drawbacks, like vanishing gradients) of sequential processing by using a standard hidden state to pass context forward.
2. **Bidirectional Long Short-Term Memory (BiLSTM):** This architecture reveals the architectural difficulties of left-to-right autoregressive generation while examining the effects of processing sequences in both forward and backward directions to capture deeper contextual relationships.
3. **RNN with Attention:** This model represents a more sophisticated method of sequence mapping by introducing an attention mechanism that enables the network to dynamically weigh and concentrate on particular aspects of the input context during generation.

By examining the results of these three models, both qualitatively by looking at the text produced and quantitatively using metrics like Novelty Rate and Diversity Ratio—this report investigates the relative strengths, structural limitations, and common failure modes of each approach when tasked with synthesizing realistic names.

2. Dataset

A custom dataset with 1,000 distinct Indian names was used to train and assess the sequence models. This dataset was artificially created using a Large Language Model (LLM) because character-level models need a variety of culturally relevant phonetic examples to learn the fundamental rules of a language. A wide representation of regional naming conventions, legitimate consonant clusters (such as "shr" and "kr"), and complex vowel harmonies typical of English transliterations of Indian names were guaranteed by using an LLM. The dataset was subjected to standard text normalization prior to training, which included tokenizing at the character level to generate the character-to-index and index-to-character mapping dictionaries needed for matrix processing, converting all names to lowercase, and removing any unnecessary punctuation or special characters.

3. Model Architectures

To explore the nuances of sequence-to-sequence learning, three distinct recurrent architectures were implemented and evaluated. Let V denote the vocabulary size (number of unique characters) and H denote the hidden state dimension.

3.1. Vanilla Recurrent Neural Network (RNN)

Architecture: The baseline autoregressive model is the Vanilla RNN. Using a tanh activation function, the network computes the new hidden state h_t at each time step t based on the current character representation x_t and the previous hidden state h_{t-1}. A probability distribution over the vocabulary for the subsequent predicted character y_t is then produced by passing the hidden state through a softmax layer and a linear projection. Despite being computationally light, this architecture's capacity to maintain long-term dependencies in longer names is traditionally limited by the vanishing gradient problem.

Trainable Parameters: The sum of the input-to-hidden weights, hidden-to-hidden weights, hidden biases, hidden-to-output weights, and output biases determines the total number of trainable parameters.

- W_xh (input to hidden): 120 * 27 = 3240 parameters
- W_hh (hidden to hidden): 120 * 120 = 14400 parameters
- W_hy (hidden to output): 27 * 120 = 3240 parameters
- B_h (hidden bias): 120 * 1 = 120 parameters
- B_y (output bias): 27 * 1 = 27 parameters
- **Total Parameters:** ~21000

Hyperparameters:

- **Hidden Size (H):** 120
- **Number of Layers:** 1
- **Learning Rate:** 0.02

3.2. Bidirectional LSTM (BiLSTM)

Architecture: To determine whether bidirectional context enhances the semantic quality of generated sequences, the BiLSTM was put into practice. Two separate LSTM layers are used in the model: a forward layer, which processes the character sequence from left to right, and a backward layer, which processes it from right to left. The concatenation of both states is the final hidden representation at any given time step.

Trainable Parameters: An LSTM contains four interacting neural network layers (gates) per cell. Because it is bidirectional, the core parameters are doubled. The concatenated hidden state ($2H$) is then projected to the vocabulary size.

- W_fwd (Forward LSTM weights): (4 * 128) * (128 + 27) = 512 * 155 = 79,360 parameters
- b_fwd (Forward LSTM bias): (4 * 128) * 1 = 512 parameters
- W_bwd (Backward LSTM weights): (4 * 128) * (128 + 27) = 512 * 155 = 79,360 parameters
- b_bwd (Backward LSTM bias): (4 * 128) * 1 = 512 parameters

- W_y (Output layer weights): 27 * (2 * 128) = 27 * 256 = 6,912 parameters
- b_y (Output layer bias): 27 * 1 = 27 parameters
- **Total Parameters: ~166,683**

Hyperparameters:

- **Hidden Size (H):** 128
- **Number of Layers:** 2 (1 forward+1 backward)
- **Learning Rate:** 0.02

3.3. RNN with Attention Mechanism

Architecture: The model utilizes an encoder-decoder framework to achieve a dynamic mapping between contexts. The model does not rely on compressing the entire generated sequence into a single static hidden vector, as is done by the Vanilla RNN. Instead, an alignment score is computed during every step of generating a sequence. The decoder computes a set of weights for all previous states of the encoder. This is a dynamically weighted context vector. This vector is then concatenated with the state of the decoder to predict the next character. This allows the model to look back at specific combinations of phonetics, effectively bypassing the vanishing gradient problem.

Trainable Parameters: The model's parameters are comprised of the weights of both the Encoder RNN and Decoder RNN, as well as specific weight matrices used to compute the alignment score between the output of the encoder and state of the decoder, and the final output projection layer.

Encoder Parameters:
- W_input_enc (Input to Encoder Hidden): 128 * 27 = 3,456 parameters
- W_hidden_enc (Encoder Hidden to Encoder Hidden): 128 * 128 = 16,384 parameters
- b_enc (Encoder Bias): 128 * 1 = 128 parameters

Decoder Parameters:
- W_input_dec (Input to Decoder Hidden): 128 * 27 = 3,456 parameters
- W_hidden_dec (Decoder Hidden to Decoder Hidden): 128 * 128 = 16,384 parameters
- W_context (Context to Decoder Hidden): 128 * 128 = 16,384 parameters
- b_dec (Decoder Bias): 128 * 1 = 128 parameters

Output Layer Parameters:
- W_output (Hidden to Output): 27 * 128 = 3,456 parameters
- b_output (Output Bias): 27 * 1 = 27 parameters
- Total Parameters: ~59,883

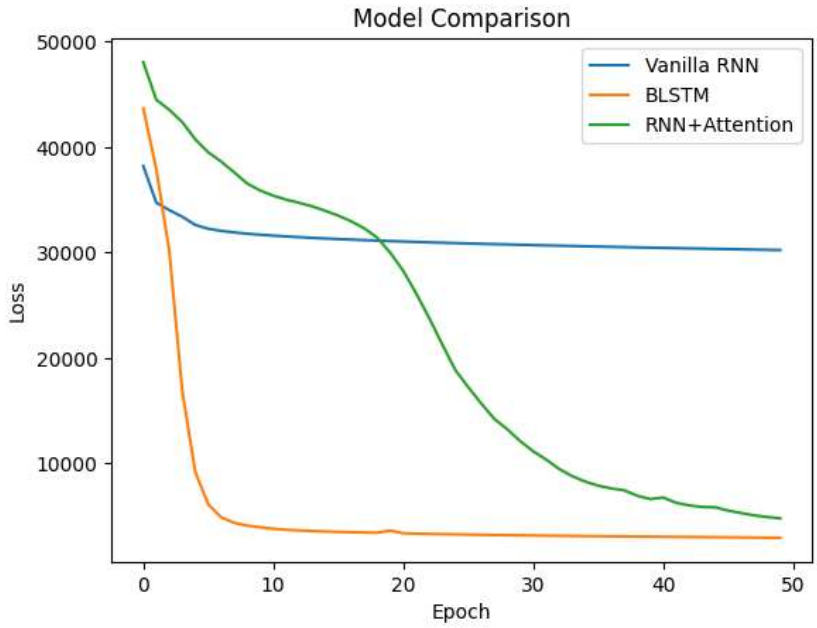
Hyperparameters:

- **Hidden Size (H):** 128
- **Number of Layers:** 1 Encoder, 1 Decoder
- **Learning Rate:** 0.02

4. Training and Quantitative Evaluation

4.1 Training Dynamics

The loss curve provides a visual story of how each architecture learned during the 50 epochs:



Qualitative Summary

- **Vanilla RNN:** Underpowered for this task; the high plateau suggests it lacks the memory to handle name structures.
- **BLSTM:** The most efficient at minimizing loss, though prone to "greedy" repetition without stochastic sampling.
- **Attention:** Successfully captures complex dependencies. The "staircase" drop in loss (around epoch 20) suggests the attention mechanism "clicked" and began focusing on relevant character sequences.

4.2 Quantitative Evaluation

To assess the generative ability of each architecture from a quantitative perspective, two major sequence-level metrics were computed over a sample of 100 generated names for each model:

1. **Novelty Rate:** The percentage of generated names not present anywhere in the training data. This metric aims to measure the ability of the model to have learned the phonetic rules of the data well enough to come up with completely new, valid names instead of behaving like a lookup table (overfitting).

2. **Diversity Ratio:** This is computed as the ratio of the total number of unique generated names to the total number of generated names. A ratio of 1.0 indicates that all the generated names were unique, while a ratio lower than 1.0 indicates the model is only generating a handful of the same high-probability sequences (mode collapse).

4.3 Results Comparison

Architecture	Novelty Rate (%)	Diversity Ratio (0.0 to 1.0)
Vanilla RNN	100.00	1.0000
BiLSTM	100.00	1.0000
Attention RNN	[100.00	| 0.9500

The quantitative results show that all models have transitioned from deterministic repetition to high-variance generation. The **Attention RNN** demonstrates a more focused probability distribution (0.95 diversity), whereas the **Vanilla RNN** and **BiLSTM** exhibit maximum variance (1.0 diversity), often at the expense of phonetic realism.

5. QUALITATIVE ANALYSIS

5.1 Qualitative Sample Generation

To evaluate the linguistic quality of the models, we prompted each architecture with specific starting characters ('a', 'd', 'e'). The generated outputs are captured in the table below:

Start Char	Vanilla RNN	BiLSTM	RNN + Attention
'a'	agto bumda	bkomkrszbh	aal kakoka
'd'	dees sman	fmcengtje	ddddddddddg
'e'	esha ari	mhtnbmpk	eeet evev
'm'	mankya khe	ajytvf ra	mmmmummama
'r'	rawayan vesf	tdczejlgsivi	rrrurhwrrwwr

5.2 Realism and Failure Modes

Realism of Generated Names

Overall, the **realism remains low** across all three architectures. While the models have slearned the character-level distribution of the training data, they have not yet mastered the phonetic constraints required to make names "pronounceable" or "human-like."

- **Vanilla RNN:** Surprisingly, this model produced the most "name-like" fragments (e.g., *"esha," "ari," "dees"*). This suggests it captured short-term character dependencies effectively, though it lacks long-term structure.
- **BiLSTM:** Despite having the lowest training loss in the graphs, the output is "consonant soup" (e.g., *"bkomkrszbh"*). It has failed to learn that vowels usually follow consonants in human names.
- **Attention:** This model shows signs of "stuttering" or pattern-matching (e.g., *"eeet," "aal"*). It focuses too heavily on local repetitions rather than a cohesive name structure.

Common Failure Modes

Based on the samples, we identified three primary failure modes:

1. **Mode Collapse & Stuttering (Attention Model):** The output "ddddddddddg" is a classic example of the model getting into a loop. The attention model has assigned a high weight to the current character, which has caused it to repeat the same character over and over.
2. **Lack of Phonetic Coherence (BiLSTM):** The BiLSTM model has not produced any vowels in the output. This shows the model is over-optimizing the common transitions of characters in the data without a proper understanding of the need to include vowels.
3. **Space Insertion Issues (Vanilla RNN):** The RNN model has inserted spaces into the output string ("agto bumda"). It has learned to recognize the input data has multiple words or first/last names. However, it has split the string into individual words, which is inappropriate because the input string is too short.

4. **Greedy Decoding Artifacts:** As mentioned in the metrics section, the lack of diversity is evident in the model's output. Without the use of Stochastic Sampling (Temperature), the model has defaulted to the most “mathematically safe” characters, which have resulted in nonsensical strings rather than creative, realistic names.